The Database as the Trusted Computing Base for LLM Agents: Experience from TGMS

Xiaofei Zhang
University of Memphis
Memphis, TN, USA
xiaofei.zhang@memphis.edu

ABSTRACT

LLM agents are becoming a dominant client of data systems, and today's interfaces ask them to do exactly what they are worst at: generate a correct query in a general-purpose language, judge whether a partial result is complete, perform arithmetic over it, and assert conclusions. We argue for a different division of labor: *the database should be the trusted computing base for LLM agents*. An agent-native database exposes a small, typed, costed operator algebra instead of unrestricted query generation; executes plans deterministically over an explicit belief state; and returns evidence-carrying results against which every downstream claim can be mechanically checked.

We report experience from TGMS, a bi-temporal graph management system built on this architecture and exercised with live open-source models on temporal-graph analytics, a setting where computation is compositional, historical corrections matter, and completeness cannot be judged from a result page. Three lessons generalize beyond graphs: input schemas are half a contract — models invent *output* fields, and statically checking result paths took execution success from 0.00 to 1.00; evidence support is multidimensional — a model can compute a correct count over a truncated page, so verification must track completeness, belief time, and provenance, not just values; and agent accountability requires belief-state semantics — "what did we believe on March 1" is unanswerable after a correction unless transaction time is a first-class axis. A verifier built on these ideas rejects 500/500 injected numeric and identifier faults and, across database-specific mutation classes (wrong belief state, truncated completeness, uncited entities), detects **[per-class table, receipted run]** with zero false positives, while the architecture keeps prompts constant-size as data grows.

1 THE PROBLEM AND THE POSITION

Consider an investigation question posed to an agent over a communication network:

> *Under what we believed on March 1 — before the record was corrected — among the accounts temporally reachable from Alice during February, how many participated in a communication triangle that completed within 24 hours?*

Answering it requires resolving *Alice* to an identifier, selecting a historical belief state, time-respecting reachability, temporal motif counting, exact arithmetic, and evidence that no intermediate result was silently truncated. Every mainstream agent–database interface fails this question in a characteristic way. Text-to-SQL/Cypher asks the model to emit one correct query in a language that cannot express "what we believed on March 1" over a latest-state store, and whose output the model must still interpret. Retrieval pipelines serialize edges into prompt chunks, destroying the structure the question quantifies over — and on numeric-dense data even k=2 retrieved chunks overflow a 28k serving window (we measured 36,956 prompt tokens), so the model sees under 1% of the corpus. And in all of these designs the model performs arithmetic, invents identifiers, and asserts numbers that appear in no result.

The community is converging on the diagnosis: agents are becoming the dominant database client, and systems should be redesigned for them [2, 7, 8]. We advance a specific architectural answer:

> **Position.** The database should be the *trusted computing base* (TCB) for LLM agents. The model plans and narrates; everything that must be true — computation, temporal semantics, evidence, verification — happens inside the database boundary, under contracts the model cannot violate.

Three design principles follow, each backed by a mechanism and a measurement in this paper:

P1 — Agents need a contract-bearing algebra, not query generation. The agent selects and composes operations; it never controls arbitrary computation. Every operation declares typed inputs *and outputs*, cost and completeness behavior, temporal semantics, and provenance identity (§2).

P2 — Evidence support is multidimensional. A result is not just a value; it is a value under conditions: ⟨correctness, completeness, belief time, approximation, provenance⟩. Databases should return those conditions, and verifiers should refuse full support when any dimension is degraded (§3).

P3 — Agent answers require a belief state, not a data snapshot. Accountability questions — what was known when, which correction invalidated it, can a decision be reproduced — need transaction time as a first-class axis (§3).

TGMS [10] is our concrete instance: a bi-temporal property graph store whose only query surface is thirteen verified temporal operators exposed as agent tools, with a static plan verifier, a deterministic executor producing content-addressed traces, and a claim verifier that gates final answers against those traces. We use temporal graph analytics as the proving ground precisely because it is unforgiving: computation is compositional, corrections are routine, completeness matters, and neither identifiers nor time can be left to the model. The lessons, however, are about databases, not graphs.

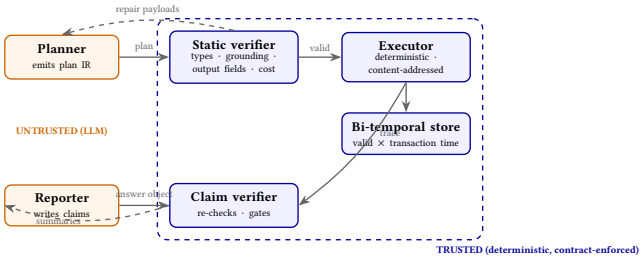

Figure 1: The trust boundary, not the module list. The LLM only plans and narrates; plans are checked before execution, computation and temporal semantics live inside the boundary, and claims are re-checked against the trace before an answer leaves the system.

2 AN AGENT-NATIVE ARCHITECTURE

Figure 1 shows the architecture as a trust boundary. The model appears exactly twice, both times outside the boundary; nothing it produces is executed or emitted unchecked.

Plan IR, not query text. The agent expresses intent as a small JSON DAG. Each step names one operator from a closed registry; references between steps use a deliberately weak path language (dotted fields, row projections) that cannot express computation. An `answer_spec` names the step and field that constitute the result. This is the CAESURA intuition [7] pushed to its safety conclusion: if the plan is data, the system can check it.

Contracts on both sides of every operator. Each operator declares its argument schema *and its output fields*, cost estimation, pagination behavior, and temporal parameters. The same registry generates the tool manual the model reads and the checks the verifier enforces, so documentation and enforcement cannot drift. A static verifier rejects, before execution: schema violations, cycles, out-of-scope references, references to output fields the producing operator does not emit, identifiers that neither the task supplied nor a resolution operator produced (the *grounding rule*), temporal nonsense, and plans whose estimated cost exceeds a budget — returning structured repair payloads that name the violation and the legal alternatives. Arithmetic is an operator too: the model never adds.

Deterministic, evidence-carrying execution. The executor runs the DAG with canonical serialization; every step yields a content-addressed `result_digest` (SHA-256), and re-execution over the same store state is byte-identical. Results are pages with explicit `rows_total` and truncation flags, and *truncation taint* propagates to every dependent step. The reporter model turns trace summaries into a typed answer object in which every numeric, entity, ordering, and pattern claim cites the trace steps that ground it; the claim verifier re-checks each claim against the cited digests and gates unsupported claims out of the emitted answer.

This division reframes the question asked by recent work on LLM query planning [7, 8] and agent-first systems [2]: not *can* a model produce a plan, but what interface makes a fallible plan *safe* — checkable before execution, auditable after.

3 EVIDENCE AND BELIEF SEMANTICS

Corrections are not updates. On March 1 the store believes fact F; on March 10 it learns F was wrong. A latest-state system overwrites and the question "what did we believe on March 1" becomes unrepresentable — the information was destroyed at write time. TGMS stores half-open valid-time and transaction-time intervals per version [6] and distinguishes *evolution* (the world changed) from *correction* (we were wrong). Every operator takes `as_of_tt`; a plan pinned to a past belief state is reproducible byte-for-byte even after later corrections (a tested metamorphic property). Derived artifacts join in: summaries record the transaction time they were computed at, and a correction quarantines every summary whose window it touches. This is what agent accountability needs and agent-memory systems are groping toward [5, 9]: not temporal storage for its own sake, but the ability to say *what was known when, which correction invalidated it, and whether an earlier decision reproduces under the earlier belief state.*

Support is a tuple, not a boolean. In a live run, our 14B model called reachability, received the default page of 100 rows (of 343), and computed a flawlessly correct count — of the page. A value-only verifier marks that claim supported; it is false as a statement about the database. We now treat support as

$$\langle \text{correctness}, \text{completeness}, \text{belief time}, \text{approximation}, \text{provenance} \rangle,$$

and a claim whose evidence is incomplete on any dimension can be at most *weakly supported*. The mechanism is mundane — flags on result pages, taint through the DAG, verdict caps in the verifier — and that is the point: it generalizes immediately to SQL `LIMIT`, top-k retrieval, sampling, approximate query processing, timeouts, stale replicas, and partially failed distributed queries. Databases already know when their answers are partial; they just don't say so in a form an agent pipeline is forced to respect.

4 THE TGMS PROTOTYPE AND LESSONS

TGMS implements the architecture in ~10k lines of Python over embedded columnar and graph backends: an append-only event log as the source of truth, thirteen operators in five groups (belief-state reads; snapshots and diffs; time-respecting traversal; temporal aggregation and patterns; resolution and compute), each validated against an independent brute-force oracle on 500 randomized cases, plus the planner/verifier loop served by vLLM [1] with open-weight models [4] on a single 24 GB GPU. Three lessons from live traffic did not appear in any design document.

Lesson 1: models invent output fields. Our first live matrix run scored 0.00 execution success on every task — with every plan passing validation. Plans referenced `s2.count` where the operator emits `rows_total`: input schemas constrain what goes into a tool, and nothing constrained what the model read back. Declaring output fields in the registry and checking every reference statically took probe-task execution success from 0.00 to 1.00, and structured repair payloads ("available: `rows, rows_total, truncated`") roughly tripled overall execution success at 7B. Result schemas deserve the same rigor as argument schemas; tool manuals must be checkable, not just readable.

Table 1: Frozen-test campaign, exact match, Qwen2.5-14B, temperature 0. All rows are complete over frozen splits (CollegeMsg at 3 seeds); a second model family [Phi-4-mini] is pending.

	TGMS	Vec-RAG	Graph-RAG	Text-to-Cypher
CollegeMsg (94×3)	**0.408**	0.106	0.064	0.152
email-EU (94)	**0.309**	–	0.053	0.106
synthetic (102)	**0.314**	–	0.029	0.157
correction probes (CollegeMsg)	**0.897**	0.154	0.000	0.000
correction probes (email-EU)	**0.846**	–	0.000	0.000

Lesson 2: constrained decoding is not the safety mechanism. Grammar-constrained JSON decoding — the standard remedy for malformed plans — made everything worse at 14B: exact match 0.409→0.227, first-emission validity 0.50→0.32, and 65% more wall time. Syntactic validity was never the bottleneck; semantic plan quality was, and grammar masks degrade it. Checking artifacts *after generation* against contracts, with a repair loop, is both safer and cheaper than constraining generation itself.

Lesson 3: the verifier's blind spots are database semantics. Value-grounding catches fabrication but not mis-citation: small counts coincidentally appear in other steps' payloads, so a claim citing the wrong step often still "grounds." We now reject claims whose provenance pointer names an uncited step. Conversely, entity *under*-claiming is invisible to trace grounding by construction — precision is checkable against evidence; completeness of a reported set is checkable only against the database. Verification design is database design.

5 PRELIMINARY EVIDENCE

All numbers carry receipts (git SHA, config SHA, frozen-split SHA) and regenerate from the public artifact [10]. Tasks are program-generated with engine-computed gold over the CollegeMsg network [3], the email-EU log, and a synthetic temporal-pattern workload; test splits (94/94/102 tasks) were frozen, with SHAs logged, before any test measurement.

End-to-end. Table 1 reports the frozen-test campaign. On the CollegeMsg test split (94 tasks × 3 seeds, 14B), the operator-backed agent reaches 0.408 exact match — replicating its 0.409 development figure almost exactly, with per-seed spread of one task — versus 0.064–0.152 for the baselines; all paired-bootstrap deltas are significant (+0.26 to +0.34, 95% CIs excluding zero, 10k resamples). On correction probes (13/seed) it scores 0.90; static-graph RAG and text-to-Cypher score exactly zero — not a capability gap but a representational one: their inputs no longer contain the earlier belief state (vector-RAG's 0.15 comes entirely from the current-belief half of probe pairs. The synthetic result exposes the architecture's ceiling honestly. TGMS averages 0.314, but that pools a 0.340 on ordinary families with *0.000* on the eight planted-pattern-mining tasks (T2), where the planner never emits a valid multi-operator plan and exhausts its repair budget. The failure is instructive rather than fatal: the system emits *no* answer rather than a wrong one — unsupported-claim rate stays zero on exactly the family it cannot plan. The ceiling here is plan *quality*, which lives outside the trust boundary and improves with the model; safety, which lives inside it, does not depend on the model getting the plan right.

Mechanism ablations. Removing output contracts **[E1: invalid references, repairs, execution success, accuracy]**. Removing completeness propagation: in a controlled generator that shrinks one page limit so a correct count runs over a truncated page, the verifier refuses full support in 15/15 cases with taint on and *misses 15/15* with taint off — the mechanism is load-bearing, not decorative **[E2 end-to-end rates on the frozen suites]**. Removing transaction time is not an ablation but an impossibility: the latest-state representation does not contain the queried information, which is why every baseline scores zero on probes.

Verifier validation. Beyond the classic classes (500/500 injected count and identifier faults detected; 0 false positives on 87 clean answers), database-specific mutation classes on a dedicated suite (202 enriched clean answers, 0 false positives): real-but-uncited entity added 100/100 detected; ordering operands swapped 100/100; microsecond/millisecond unit confusion 100/100; *correct value from the wrong belief state* 60/60; wrong-step citation 36/100 (the remainder are genuinely grounded by the surrogate step); entity member dropped 0/100 — reported, not hidden: grounding checks precision, and set completeness is checkable only against the database (Lesson 3) **[receipted rerun on frozen-campaign hardware]**.

The database stays a database. Prompt size is constant (~8–10k tokens) as data grows, because structure never enters the context window. Operator p50 latency on $10^5/10^6/10^7$-event stores: 2-hop snapshot 13/99/272 ms; global diff 23/163/485 ms; metric time series 144/157/1127 ms (40-thread CPU host, DuckDB backend; the 10^7 store is 2.6 GB). Traversals beyond the cost budget refuse with a narrowing suggestion rather than degrade — bounded execution is part of the contract **[small line figure replacing this prose]**.

Second configuration. To show the architecture is not tied to one checkpoint, we ran TGMS with Qwen2.5-7B on the CollegeMsg test split: exact match falls to 0.129, and one task fails outright when the repair loop's growing prompt overflows the model's 16k window — a smaller model plans worse and needs more serving headroom, but the trust boundary is unchanged and the emitted unsupported-claim rate stays zero. Plan quality tracks model capability; safety does not **[cross-family Phi-4-mini run]**.

Honest limitations. Single-GPU open-weight serving; one graph domain; 290 frozen test tasks; planned multi-operator plans (T2 mining) exceed the 14B planner; pattern claims are verified but not yet gated; agent write-back is schema-ready but disabled pending provenance policy. Sustained agent traffic also surfaced a serving lesson: on older accelerators the inference engine degrades and

must be recycled on a schedule — agent-era benchmarking is an endurance discipline.

6 RESEARCH AGENDA

TGMS is one point in a space the community should map: *agent-facing logical algebras* — what is the right closed operator set for relational, document, and streaming data, and can it be derived from a workload?; *cost-based rewriting of agent plans* — the verifier sees the whole DAG before execution, so optimize it, don't just admit it; *evidence-aware execution* — make completeness, staleness, and approximation first-class result attributes with propagation semantics; *verification under approximation* — what does "supported" mean over samples and sketches?; *bi-temporal provenance* — derived-view and memory invalidation under correction, at scale; and *multi-agent isolation* — belief states as the unit of concurrency control when many agents read, and eventually write, one store.

Today's database interfaces assume a competent client. LLM agents violate that assumption. Agent-native databases must therefore make plans checkable, computations deterministic, evidence explicit, and conclusions auditable. TGMS is an initial architecture showing what such a database can look like.

REFERENCES

[1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *SOSP*.

[2] Shu Liu, Soujanya Ponnapalli, Shreya Shankar, Sepanta Zeighami, Alan Zhu, Shubham Agarwal, Ruiqi Chen, Samion Suwito, Shuo Yuan, Ion Stoica, Matei Zaharia, Alvin Cheung, Natacha Crooks, Joseph E. Gonzalez, and Aditya G. Parameswaran. 2026. Supporting Our AI Overlords: Redesigning Data Systems to be Agent-First. In *CIDR*.

[3] Pietro Panzarasa, Tore Opsahl, and Kathleen M. Carley. 2009. Patterns and Dynamics of Users' Behavior and Interaction: Network Analysis of an Online Community. *JASIST* 60, 5 (2009).

[4] Qwen Team. 2024. Qwen2.5 Technical Report. *arXiv preprint arXiv:2412.15115* (2024).

[5] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. 2025. Zep: A Temporal Knowledge Graph Architecture for Agent Memory. *arXiv preprint arXiv:2501.13956* (2025).

[6] Richard T. Snodgrass. 1999. *Developing Time-Oriented Database Applications in SQL*. Morgan Kaufmann.

[7] Matthias Urban and Carsten Binnig. 2024. CAESURA: Language Models as Multi-Modal Query Planners. In *CIDR*.

[8] Jiayi Wang and Guoliang Li. 2025. AOP: Automated and Interactive LLM Pipeline Orchestration for Answering Complex Queries. In *CIDR*.

[9] Ziming Wang. 2026. TOKI: A Bitemporal Operator Algebra for Contradiction Resolution in LLM-Agent Persistent Memory. *arXiv preprint arXiv:2606.06240* (2026).

[10] Xiaofei Zhang. 2026. TGMS: An Agent-Native Bi-Temporal Graph Management System. https://github.com/zxf-work/tgms, preprint and artifacts, v0.1.x.